

Tema 7: Grafer

Wilhelm Durelius widu7139

22 februari 2026

1 Diskussion

Vid genomgång av de olika typerna av representationer, fastnade jag för en samling av bågar. Det kändes som ett väldigt simpelt sätt att hantera grafen på. Det visade sig dock att då tappar man viss funktionalitet, det är svårt att hitta specifika noder eller bågar. Speciellt i en oriktad graf fick jag känslan att detta skapar mer problem än enkelheten i det löser. Bland annat behöver man då spara data dubbelt, utan referenser, eller implementera en mer komplex sökalgoritm som automatiskt alltid söker åt båda hållen.

För att kunna spara referenser till data istället för själva datan, valde jag att istället gå med en nodklass. Det enda som blir dubletter i minnet är då kostnaden för bågarna i en oriktad graf, då jag hellre sparade bågar åt båda hållen för att effektivisera lookup-tid, än att spara bara åt ena hållet och behöva leta igenom alla bågar för att skapa par.

2 Nodklass med hashtabell

En **Nodklass och Hashtabell** kombination där varje nod lagrar en lista med utgående kanter direkt i nodobjektet, samt nodens data av godtycklig typ. Varje kant lagrar referenser till där den kom ifrån, och dit den ska, samt kostnaden. Även om det är en hashtabell, kan vi komma åt alla element i tabellen på samma sätt som en lista om vi skulle behöva. Därför kallas denna variant **Adjacency List**.

Då vi inte tillåter dubletter i grafen skapar vi en hash baserat på datan som ska in i nodobjektet, och använder det som nyckel för att hitta objektet i vår map. Detta ger oss $O(1)$ lookup från data till nodobjekt som äger datan.

En negativ aspekt av denna struktur är att det tar längre tid att hitta en specifik kant. Om vi vet att vi ska hitta kanten mellan nod A och nod B, och att det finns en sådan kant, måste vi fortfarande iterera genom listan med kanter på noden. Ska vi till exempel ändra kostnaden på kanten, och det är en oriktad graf, så behöver vi i denna representation av grafen också hitta kanten från nod B till nod A. Det är inte en nästlad for loop, så det blir $O(e_1 + e_2)$ där e är antalet kanter per nod.

Man brukar använda en adjacency list när antalet kanter är mindre än v^2 [1] (v står för Vertices, betyder noder). Adjacency List lagrar för varje nod endast de kanter som faktiskt finns. Minnesanvändningen är $O(v + e)$ där m är antalet kanter, vilket är optimalt för glesa grafer.

Med en Adjacency List besöker både DFS och BFS varje nod exakt en gång och följer varje kant exakt en gång, vilket ger tidskomplexiteten $O(v + e)$. Minneskomplexiteten är $O(v)$

för den besökta mängden och listan, plus $O(v + e)$ för själva grafen.

Med en kopplingsmatris däremot måste man för varje nod iterera över hela raden i matrisen för att hitta nästa nod, oavsett hur många noder som faktiskt finns. Detta ger tidskomplexiteten $O(v^2)$ för båda algoritmerna. För glesa grafer (e är mindre än v^2) är detta en betydande försämring. Minnesanvändningen för matrisen är dessutom alltid $O(v^2)$, oavsett grafens faktiska täthet.

Adjacency List är därmed överlägsen för glesa grafer, vilket är det vanligaste fallet för all dagliga utvecklare på DSV. Kopplingsmatris blir aktuell när grafen är väldigt tät och man ofta vill komma åt enskilda kanter direkt.

Referenser

- [1] AlgoCademy. When to use an adjacency list vs matrix: A comprehensive guide. <https://algotcademy.com/blog/when-to-use-an-adjacency-list-vs-matrix-a-comprehensive-guide/>. Accessed: 2026-02-22.